Internet Programming CSA4302

LAB EXPERIMENTS 01–10

Shivani S 192471030

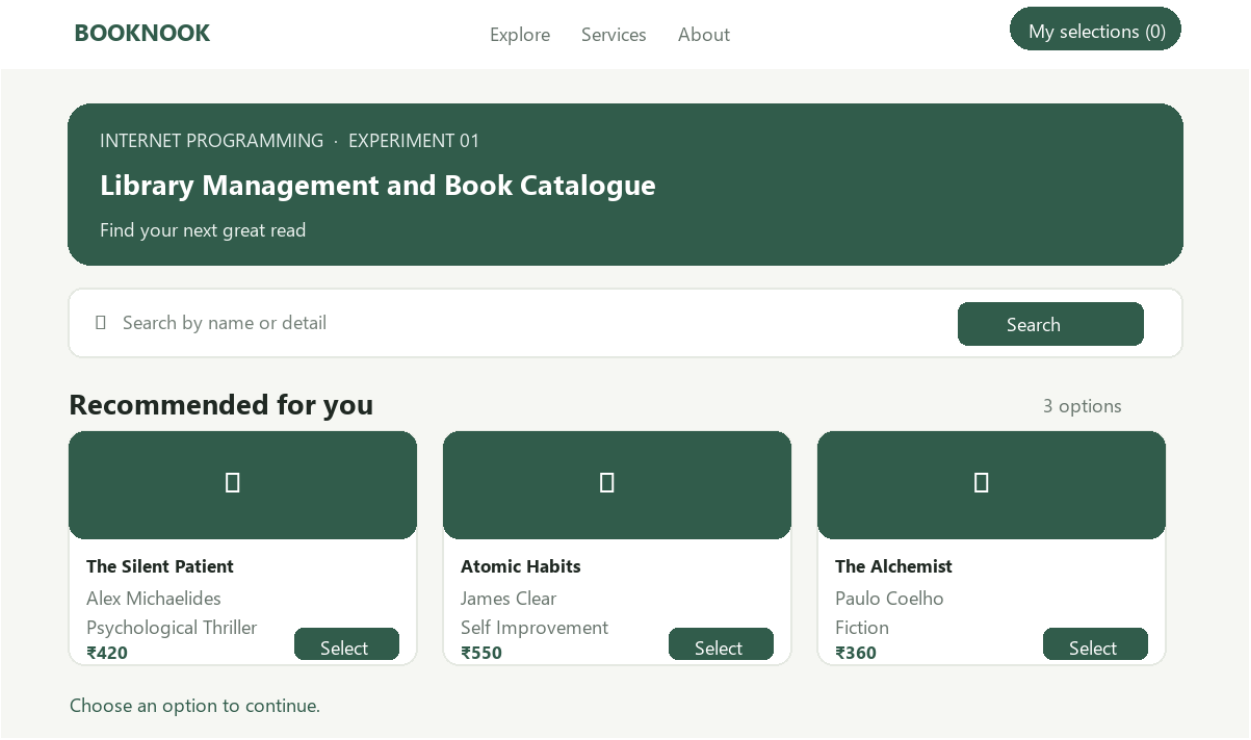
Experiment Index

Ten responsive front-end demonstrations based on the first ten project briefs in the lab reference.

Experiment 1: Library Management and Book Catalogue

Aim: To design a responsive library portal that lets students search the book catalogue, view availability, and request a book, while presenting librarian inventory controls.

Output screenshot



Result: The library catalogue was displayed with sample titles and supports searching and selecting a book.

Complete source code (standalone HTML)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Library Management and Book Catalogue</title>
  <style>
    :root {
      --ink: #202923;
      --muted: #68756d;
      --paper: #f6f7f3;
      --card: #fff;
      --line: #e5e9e2;
      --accent: #315c4b;
    }
    * { box-sizing: border-box; }
    body {
      margin: 0;
      background: var(--paper);
      color: var(--ink);
      font: 15px/1.5 system-ui, -apple-system, "Segoe UI", sans-serif;
    }
    header {
      display: flex;
      align-items: center;
      justify-content: space-between;
      gap: 20px;
```

```
      padding: 16px max(5vw, 20px);
      background: #fff;
      border-bottom: 1px solid var(--line);
    }
    .brand { color: var(--accent); font-size: 18px; font-weight: 800; letter-spacing: .12em; }
    nav { display: flex; gap: 22px; color: var(--muted); font-size: 14px; }
    .cart, .action {
      border: 0;
      background: var(--accent);
      color: #fff;
      font-weight: 700;
      cursor: pointer;
    }
    .cart { border-radius: 99px; padding: 10px 16px; }
    main { max-width: 1100px; margin: 40px auto; padding: 0 22px; }
    .hero {
      padding: 30px 32px;
      border-radius: 22px;
      color: #fff;
      background: linear-gradient(120deg, var(--accent), #273744);
    }
    .eyebrow { text-transform: uppercase; letter-spacing: .14em; font-size: 11px; opacity: .8; }
    h1 { margin: 7px 0 8px; font-size: clamp(28px, 4vw, 42px); line-height: 1.1; }
    .hero p { margin: 0; opacity: .88; }
    .search {
      display: flex;
      flex-wrap: wrap;
      gap: 10px;
      margin: 20px 0 24px;
      padding: 14px;
      border: 1px solid var(--line);
      border-radius: 16px;
      background: #fff;
    }
    input, select {
      min-width: 150px;
      flex: 1;
      padding: 11px 12px;
      border: 1px solid var(--line);
      border-radius: 10px;
      color: var(--ink);
      background: #fff;
      font: inherit;
    }
    .sectionhead {
      display: flex;
      align-items: end;
      justify-content: space-between;
      margin: 26px 0 14px;
    }
    h2 { margin: 0; font-size: 22px; }
    .sectionhead span { color: var(--muted); font-size: 13px; }
    .grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(245px, 1fr)); gap: 16px; }
    .card {
      overflow: hidden;
      border: 1px solid var(--line);
      border-radius: 16px;
      background: var(--card);
      box-shadow: 0 5px 18px #2233220a;
    }
    .art {
      display: grid;
      place-items: center;
      height: 125px;
      color: #fff;
      background: linear-gradient(135deg, var(--accent), #d9b79a);
      font-size: 34px;
    }
    .body { padding: 16px; }
    .body h3 { margin: 0 0 5px; font-size: 17px; }
    .body p { margin: 3px 0; color: var(--muted); font-size: 13px; }
    .row {
      display: flex;
      align-items: center;
      justify-content: space-between;
      gap: 10px;
      margin-top: 14px;
    }
    .price { color: var(--accent); font-weight: 800; }
    .action { border-radius: 9px; padding: 9px 12px; }
    .notice { min-height: 24px; margin: 16px 2px; color: var(--accent); font-weight: 700; }
    footer {
      margin-top: 35px;
      padding: 18px 4px;
      border-top: 1px solid var(--line);
      color: var(--muted);
      font-size: 12px;
    }
    @media (max-width: 600px) {
      header { align-items: flex-start; flex-wrap: wrap; }
      nav { order: 3; width: 100%; gap: 12px; flex-wrap: wrap; }
      main { margin: 22px auto; }
      .hero { padding: 24px; }
    }
  </style>
</head>
<body>
```

```
  <header>
    <div class="brand">BOOKNOOK</div>
    <nav><span>Explore</span><span>Services</span><span>About</span></nav>
    <button class="cart" id="cartButton">My selections (0)</button>
  </header>
  <main>
    <section class="hero">
      <div class="eyebrow">Internet Programming · Experiment 01</div>
      <h1>Library Management and Book Catalogue</h1>
      <p>Find your next great read</p>
    </section>
    <section class="search" aria-label="Search and filters">
      <input id="searchBox" type="search" placeholder="Search by name or detail" aria-label="Search">
      <select id="category">
        <option value="">All options</option>
        <option value="popular">Popular</option>
        <option value="available">Available now</option>
      </select>
      <button class="action" id="searchButton">Search</button>
    </section>
    <div class="sectionhead">
      <h2>Recommended for you</h2>
      <span id="countLabel">3 options</span>
    </div>
    <section class="grid" id="catalog" aria-live="polite"></section>
    <div class="notice" id="notice" role="status">Choose an option to continue.</div>
    <footer>
      Demonstration interface with sample data. Authentication, live availability, payments and
      notifications require a server integration.
    </footer>
  </main>
  <script>
    const items = [
      {
        name: "The Silent Patient",
        line: "Alex Michaelides",
        meta: "Psychological Thriller",
        price: "■420"
      },
      {
        name: "Atomic Habits",
        line: "James Clear",
        meta: "Self Improvement",
        price: "■550"
      },
      {
        name: "The Alchemist",
        line: "Paulo Coelho",
        meta: "Fiction",
        price: "■360"
      }
    ];
    let selected = 0;
    const catalog = document.querySelector('#catalog');
    const notice = document.querySelector('#notice');

    function render(list = items) {
      catalog.innerHTML = '';
      document.querySelector('#countLabel').textContent = `${list.length} options`;
      list.forEach((item, index) => {
        const card = document.createElement('article');
        card.className = 'card';
        card.innerHTML = `
          <div class="art" aria-hidden="true">${['❖', '■', '✧'][index % 3]}</div>
          <div class="body">
            <h3>${item.name}</h3>
            <p>${item.line}</p>
            <p>${item.meta}</p>
            <div class="row">
              <span class="price">${item.price}</span>
              <button class="action" data-index="${index}">Select</button>
            </div>
          </div>`;
        catalog.append(card);
      });
    }

    document.querySelector('#searchButton').addEventListener('click', () => {
      const query = document.querySelector('#searchBox').value.trim().toLowerCase();
      const matches = items.filter(item =>
        `${item.name} ${item.line} ${item.meta}`.toLowerCase().includes(query)
      );
      render(matches);
      notice.textContent = query ? 'Search results updated.' : 'Showing all options.';
    });

    catalog.addEventListener('click', event => {
      const button = event.target.closest('button[data-index]');
      if (!button) return;
      const chosen = catalog.querySelectorAll('button[data-index]')[Number(button.dataset.index)];
      selected += 1;
      document.querySelector('#cartButton').textContent = `My selections (${selected})`;
      notice.textContent = `${chosen.closest('.body').querySelector('h3').textContent} added to your
selections.`;
    });

    document.querySelector('#cartButton').addEventListener('click', () => {
```

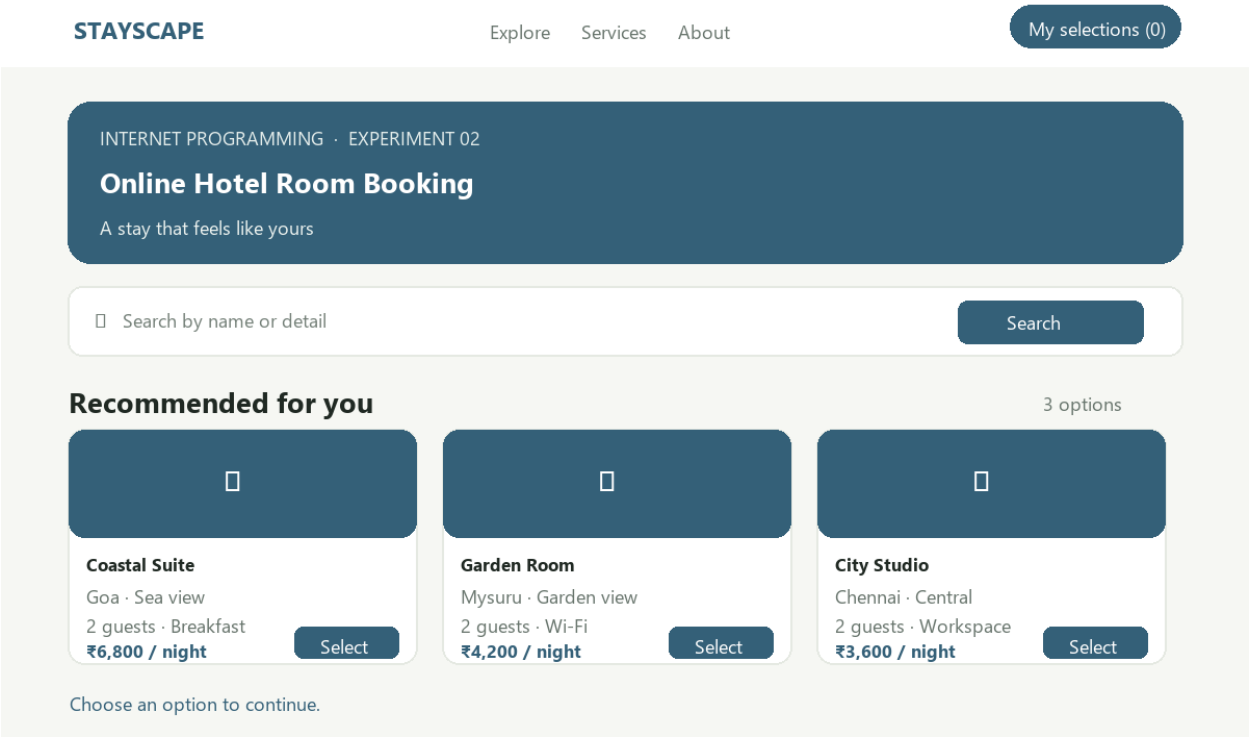
```
      notice.textContent = selected
        ? `You have ${selected} item(s) selected. Review your request.`
        : 'Your selections are empty. Choose an option first.';
    });

    render();
  </script>
</body>
</html>
```

Experiment 2: Online Hotel Room Booking

Aim: To build a hotel booking interface for searching stays by destination and dates, comparing room details and prices, and confirming a sample reservation.

Output screenshot



Result: The hotel listings and sample rates were displayed, and a guest can select a room to continue a demo booking.

Complete source code (standalone HTML)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Online Hotel Room Booking</title>
  <style>
    :root {
      --ink: #202923;
      --muted: #68756d;
      --paper: #f6f7f3;
      --card: #fff;
      --line: #e5e9e2;
      --accent: #346078;
    }
    * { box-sizing: border-box; }
    body {
      margin: 0;
      background: var(--paper);
      color: var(--ink);
      font: 15px/1.5 system-ui, -apple-system, "Segoe UI", sans-serif;
    }
    header {
      display: flex;
      align-items: center;
      justify-content: space-between;
      gap: 20px;
      padding: 16px max(5vw, 20px);
      background: #fff;
      border-bottom: 1px solid var(--line);
    }
    .brand { color: var(--accent); font-size: 18px; font-weight: 800; letter-spacing: .12em; }
    nav { display: flex; gap: 22px; color: var(--muted); font-size: 14px; }
    .cart, .action {
```

```
      border: 0;
      background: var(--accent);
      color: #fff;
      font-weight: 700;
      cursor: pointer;
    }
    .cart { border-radius: 99px; padding: 10px 16px; }
    main { max-width: 1100px; margin: 40px auto; padding: 0 22px; }
    .hero {
      padding: 30px 32px;
      border-radius: 22px;
      color: #fff;
      background: linear-gradient(120deg, var(--accent), #273744);
    }
    .eyebrow { text-transform: uppercase; letter-spacing: .14em; font-size: 11px; opacity: .8; }
    h1 { margin: 7px 0 8px; font-size: clamp(28px, 4vw, 42px); line-height: 1.1; }
    .hero p { margin: 0; opacity: .88; }
    .search {
      display: flex;
      flex-wrap: wrap;
      gap: 10px;
      margin: 20px 0 24px;
      padding: 14px;
      border: 1px solid var(--line);
      border-radius: 16px;
      background: #fff;
    }
    input, select {
      min-width: 150px;
      flex: 1;
      padding: 11px 12px;
      border: 1px solid var(--line);
      border-radius: 10px;
      color: var(--ink);
      background: #fff;
      font: inherit;
    }
    .sectionhead {
      display: flex;
      align-items: end;
      justify-content: space-between;
      margin: 26px 0 14px;
    }
    h2 { margin: 0; font-size: 22px; }
    .sectionhead span { color: var(--muted); font-size: 13px; }
    .grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(245px, 1fr)); gap: 16px; }
    .card {
      overflow: hidden;
      border: 1px solid var(--line);
      border-radius: 16px;
      background: var(--card);
      box-shadow: 0 5px 18px #2233220a;
    }
    .art {
      display: grid;
      place-items: center;
      height: 125px;
      color: #fff;
      background: linear-gradient(135deg, var(--accent), #d9b79a);
      font-size: 34px;
    }
    .body { padding: 16px; }
    .body h3 { margin: 0 0 5px; font-size: 17px; }
    .body p { margin: 3px 0; color: var(--muted); font-size: 13px; }
    .row {
      display: flex;
      align-items: center;
      justify-content: space-between;
      gap: 10px;
      margin-top: 14px;
    }
    .price { color: var(--accent); font-weight: 800; }
    .action { border-radius: 9px; padding: 9px 12px; }
    .notice { min-height: 24px; margin: 16px 2px; color: var(--accent); font-weight: 700; }
    footer {
      margin-top: 35px;
      padding: 18px 4px;
      border-top: 1px solid var(--line);
      color: var(--muted);
      font-size: 12px;
    }
    @media (max-width: 600px) {
      header { align-items: flex-start; flex-wrap: wrap; }
      nav { order: 3; width: 100%; gap: 12px; flex-wrap: wrap; }
      main { margin: 22px auto; }
      .hero { padding: 24px; }
    }
  </style>
</head>
<body>
  <header>
    <div class="brand">STAYSCAPE</div>
    <nav><span>Explore</span><span>Services</span><span>About</span></nav>
    <button class="cart" id="cartButton">My selections (0)</button>
  </header>
  <main>
    <section class="hero">
```

```
      <div class="eyebrow">Internet Programming · Experiment 02</div>
      <h1>Online Hotel Room Booking</h1>
      <p>A stay that feels like yours</p>
    </section>
    <section class="search" aria-label="Search and filters">
      <input id="searchBox" type="search" placeholder="Search by name or detail" aria-label="Search">
      <select id="category">
        <option value="">All options</option>
        <option value="popular">Popular</option>
        <option value="available">Available now</option>
      </select>
      <button class="action" id="searchButton">Search</button>
    </section>
    <div class="sectionhead">
      <h2>Recommended for you</h2>
      <span id="countLabel">3 options</span>
    </div>
    <section class="grid" id="catalog" aria-live="polite"></section>
    <div class="notice" id="notice" role="status">Choose an option to continue.</div>
    <footer>
      Demonstration interface with sample data. Authentication, live availability, payments and
      notifications require a server integration.
    </footer>
  </main>
  <script>
    const items = [
      {
        name: "Coastal Suite",
        line: "Goa · Sea view",
        meta: "2 guests · Breakfast",
        price: "■6,800 / night"
      },
      {
        name: "Garden Room",
        line: "Mysuru · Garden view",
        meta: "2 guests · Wi-Fi",
        price: "■4,200 / night"
      },
      {
        name: "City Studio",
        line: "Chennai · Central",
        meta: "2 guests · Workspace",
        price: "■3,600 / night"
      }
    ];
    let selected = 0;
    const catalog = document.querySelector('#catalog');
    const notice = document.querySelector('#notice');

    function render(list = items) {
      catalog.innerHTML = '';
      document.querySelector('#countLabel').textContent = `${list.length} options`;
      list.forEach((item, index) => {
        const card = document.createElement('article');
        card.className = 'card';
        card.innerHTML = `
          <div class="art" aria-hidden="true">${['❖', '■', '✧'][index % 3]}</div>
          <div class="body">
            <h3>${item.name}</h3>
            <p>${item.line}</p>
            <p>${item.meta}</p>
            <div class="row">
              <span class="price">${item.price}</span>
              <button class="action" data-index="${index}">Select</button>
            </div>
          </div>`;
        catalog.append(card);
      });
    }

    document.querySelector('#searchButton').addEventListener('click', () => {
      const query = document.querySelector('#searchBox').value.trim().toLowerCase();
      const matches = items.filter(item =>
        `${item.name} ${item.line} ${item.meta}`.toLowerCase().includes(query)
      );
      render(matches);
      notice.textContent = query ? 'Search results updated.' : 'Showing all options.';
    });

    catalog.addEventListener('click', event => {
      const button = event.target.closest('button[data-index]');
      if (!button) return;
      const chosen = catalog.querySelectorAll('button[data-index]')[Number(button.dataset.index)];
      selected += 1;
      document.querySelector('#cartButton').textContent = `My selections (${selected})`;
      notice.textContent = `${chosen.closest('.body').querySelector('h3').textContent} added to your
selections.`;
    });

    document.querySelector('#cartButton').addEventListener('click', () => {
      notice.textContent = selected
        ? `You have ${selected} item(s) selected. Review your request.`
        : 'Your selections are empty. Choose an option first.';
    });

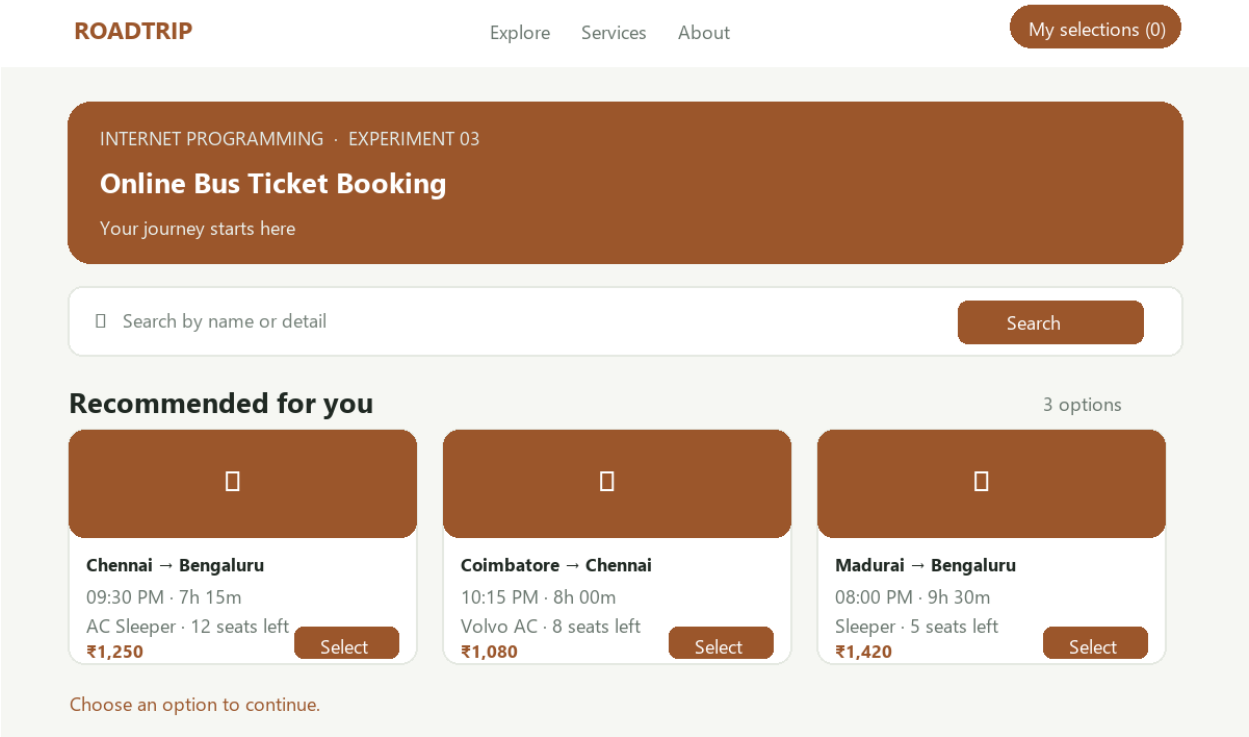
    render();
  </script>
```

```
</body>
</html>
```

Experiment 3: Online Bus Ticket Booking

Aim: To create a bus reservation page that supports route search, displays scheduled services and seat availability, and lets a passenger select a service and review a booking.

Output screenshot



Result: The sample bus services, routes, fares and seat counts were displayed, with route search and service selection available.

Complete source code (standalone HTML)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Online Bus Ticket Booking</title>
  <style>
    :root {
      --ink: #202923;
      --muted: #68756d;
      --paper: #f6f7f3;
      --card: #fff;
      --line: #e5e9e2;
      --accent: #9b562b;
    }
    * { box-sizing: border-box; }
    body {
      margin: 0;
      background: var(--paper);
      color: var(--ink);
      font: 15px/1.5 system-ui, -apple-system, "Segoe UI", sans-serif;
    }
    header {
      display: flex;
      align-items: center;
      justify-content: space-between;
      gap: 20px;
      padding: 16px max(5vw, 20px);
      background: #fff;
      border-bottom: 1px solid var(--line);
    }
    .brand { color: var(--accent); font-size: 18px; font-weight: 800; letter-spacing: .12em; }
    nav { display: flex; gap: 22px; color: var(--muted); font-size: 14px; }
    .cart, .action {
```

```
      border: 0;
      background: var(--accent);
      color: #fff;
      font-weight: 700;
      cursor: pointer;
    }
    .cart { border-radius: 99px; padding: 10px 16px; }
    main { max-width: 1100px; margin: 40px auto; padding: 0 22px; }
    .hero {
      padding: 30px 32px;
      border-radius: 22px;
      color: #fff;
      background: linear-gradient(120deg, var(--accent), #273744);
    }
    .eyebrow { text-transform: uppercase; letter-spacing: .14em; font-size: 11px; opacity: .8; }
    h1 { margin: 7px 0 8px; font-size: clamp(28px, 4vw, 42px); line-height: 1.1; }
    .hero p { margin: 0; opacity: .88; }
    .search {
      display: flex;
      flex-wrap: wrap;
      gap: 10px;
      margin: 20px 0 24px;
      padding: 14px;
      border: 1px solid var(--line);
      border-radius: 16px;
      background: #fff;
    }
    input, select {
      min-width: 150px;
      flex: 1;
      padding: 11px 12px;
      border: 1px solid var(--line);
      border-radius: 10px;
      color: var(--ink);
      background: #fff;
      font: inherit;
    }
    .sectionhead {
      display: flex;
      align-items: end;
      justify-content: space-between;
      margin: 26px 0 14px;
    }
    h2 { margin: 0; font-size: 22px; }
    .sectionhead span { color: var(--muted); font-size: 13px; }
    .grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(245px, 1fr)); gap: 16px; }
    .card {
      overflow: hidden;
      border: 1px solid var(--line);
      border-radius: 16px;
      background: var(--card);
      box-shadow: 0 5px 18px #2233220a;
    }
    .art {
      display: grid;
      place-items: center;
      height: 125px;
      color: #fff;
      background: linear-gradient(135deg, var(--accent), #d9b79a);
      font-size: 34px;
    }
    .body { padding: 16px; }
    .body h3 { margin: 0 0 5px; font-size: 17px; }
    .body p { margin: 3px 0; color: var(--muted); font-size: 13px; }
    .row {
      display: flex;
      align-items: center;
      justify-content: space-between;
      gap: 10px;
      margin-top: 14px;
    }
    .price { color: var(--accent); font-weight: 800; }
    .action { border-radius: 9px; padding: 9px 12px; }
    .notice { min-height: 24px; margin: 16px 2px; color: var(--accent); font-weight: 700; }
    footer {
      margin-top: 35px;
      padding: 18px 4px;
      border-top: 1px solid var(--line);
      color: var(--muted);
      font-size: 12px;
    }
    @media (max-width: 600px) {
      header { align-items: flex-start; flex-wrap: wrap; }
      nav { order: 3; width: 100%; gap: 12px; flex-wrap: wrap; }
      main { margin: 22px auto; }
      .hero { padding: 24px; }
    }
  </style>
</head>
<body>
  <header>
    <div class="brand">ROADTRIP</div>
    <nav><span>Explore</span><span>Services</span><span>About</span></nav>
    <button class="cart" id="cartButton">My selections (0)</button>
  </header>
  <main>
    <section class="hero">
```

```
      <div class="eyebrow">Internet Programming · Experiment 03</div>
      <h1>Online Bus Ticket Booking</h1>
      <p>Your journey starts here</p>
    </section>
    <section class="search" aria-label="Search and filters">
      <input id="searchBox" type="search" placeholder="Search by name or detail" aria-label="Search">
      <select id="category">
        <option value="">All options</option>
        <option value="popular">Popular</option>
        <option value="available">Available now</option>
      </select>
      <button class="action" id="searchButton">Search</button>
    </section>
    <div class="sectionhead">
      <h2>Recommended for you</h2>
      <span id="countLabel">3 options</span>
    </div>
    <section class="grid" id="catalog" aria-live="polite"></section>
    <div class="notice" id="notice" role="status">Choose an option to continue.</div>
    <footer>
      Demonstration interface with sample data. Authentication, live availability, payments and
      notifications require a server integration.
    </footer>
  </main>
  <script>
    const items = [
      {
        name: "Chennai → Bengaluru",
        line: "09:30 PM · 7h 15m",
        meta: "AC Sleeper · 12 seats left",
        price: "■1,250"
      },
      {
        name: "Coimbatore → Chennai",
        line: "10:15 PM · 8h 00m",
        meta: "Volvo AC · 8 seats left",
        price: "■1,080"
      },
      {
        name: "Madurai → Bengaluru",
        line: "08:00 PM · 9h 30m",
        meta: "Sleeper · 5 seats left",
        price: "■1,420"
      }
    ];
    let selected = 0;
    const catalog = document.querySelector('#catalog');
    const notice = document.querySelector('#notice');

    function render(list = items) {
      catalog.innerHTML = '';
      document.querySelector('#countLabel').textContent = `${list.length} options`;
      list.forEach((item, index) => {
        const card = document.createElement('article');
        card.className = 'card';
        card.innerHTML = `
          <div class="art" aria-hidden="true">${['❖', '■', '✧'][index % 3]}</div>
          <div class="body">
            <h3>${item.name}</h3>
            <p>${item.line}</p>
            <p>${item.meta}</p>
            <div class="row">
              <span class="price">${item.price}</span>
              <button class="action" data-index="${index}">Select</button>
            </div>
          </div>`;
        catalog.append(card);
      });
    }

    document.querySelector('#searchButton').addEventListener('click', () => {
      const query = document.querySelector('#searchBox').value.trim().toLowerCase();
      const matches = items.filter(item =>
        `${item.name} ${item.line} ${item.meta}`.toLowerCase().includes(query)
      );
      render(matches);
      notice.textContent = query ? 'Search results updated.' : 'Showing all options.';
    });

    catalog.addEventListener('click', event => {
      const button = event.target.closest('button[data-index]');
      if (!button) return;
      const chosen = catalog.querySelectorAll('button[data-index]')[Number(button.dataset.index)];
      selected += 1;
      document.querySelector('#cartButton').textContent = `My selections (${selected})`;
      notice.textContent = `${chosen.closest('.body').querySelector('h3').textContent} added to your
selections.`;
    });

    document.querySelector('#cartButton').addEventListener('click', () => {
      notice.textContent = selected
        ? `You have ${selected} item(s) selected. Review your request.`
        : 'Your selections are empty. Choose an option first.';
    });

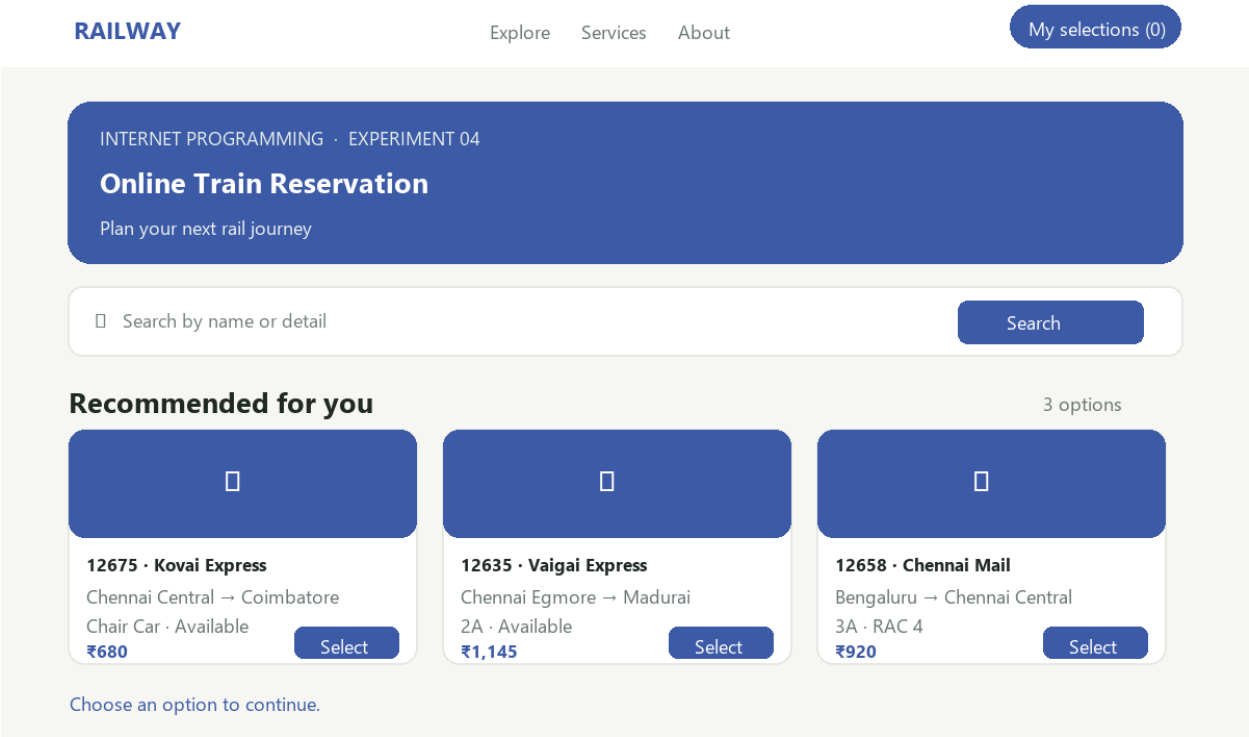
    render();
  </script>
```

```
</body>
</html>
```

Experiment 4: Online Train Reservation

Aim: To design a train reservation interface for searching trains by stations and date, viewing class availability and fares, and displaying a sample reservation reference.

Output screenshot



Result: The train options, classes, sample availability and fares were displayed, and a passenger can select a service.

Complete source code (standalone HTML)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Online Train Reservation</title>
  <style>
    :root {
      --ink: #202923;
      --muted: #68756d;
      --paper: #f6f7f3;
      --card: #fff;
      --line: #e5e9e2;
      --accent: #3c5aa6;
    }
    * { box-sizing: border-box; }
    body {
      margin: 0;
      background: var(--paper);
      color: var(--ink);
      font: 15px/1.5 system-ui, -apple-system, "Segoe UI", sans-serif;
    }
    header {
      display: flex;
      align-items: center;
      justify-content: space-between;
      gap: 20px;
      padding: 16px max(5vw, 20px);
      background: #fff;
      border-bottom: 1px solid var(--line);
    }
    .brand { color: var(--accent); font-size: 18px; font-weight: 800; letter-spacing: .12em; }
    nav { display: flex; gap: 22px; color: var(--muted); font-size: 14px; }
    .cart, .action {
```

```
      border: 0;
      background: var(--accent);
      color: #fff;
      font-weight: 700;
      cursor: pointer;
    }
    .cart { border-radius: 99px; padding: 10px 16px; }
    main { max-width: 1100px; margin: 40px auto; padding: 0 22px; }
    .hero {
      padding: 30px 32px;
      border-radius: 22px;
      color: #fff;
      background: linear-gradient(120deg, var(--accent), #273744);
    }
    .eyebrow { text-transform: uppercase; letter-spacing: .14em; font-size: 11px; opacity: .8; }
    h1 { margin: 7px 0 8px; font-size: clamp(28px, 4vw, 42px); line-height: 1.1; }
    .hero p { margin: 0; opacity: .88; }
    .search {
      display: flex;
      flex-wrap: wrap;
      gap: 10px;
      margin: 20px 0 24px;
      padding: 14px;
      border: 1px solid var(--line);
      border-radius: 16px;
      background: #fff;
    }
    input, select {
      min-width: 150px;
      flex: 1;
      padding: 11px 12px;
      border: 1px solid var(--line);
      border-radius: 10px;
      color: var(--ink);
      background: #fff;
      font: inherit;
    }
    .sectionhead {
      display: flex;
      align-items: end;
      justify-content: space-between;
      margin: 26px 0 14px;
    }
    h2 { margin: 0; font-size: 22px; }
    .sectionhead span { color: var(--muted); font-size: 13px; }
    .grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(245px, 1fr)); gap: 16px; }
    .card {
      overflow: hidden;
      border: 1px solid var(--line);
      border-radius: 16px;
      background: var(--card);
      box-shadow: 0 5px 18px #2233220a;
    }
    .art {
      display: grid;
      place-items: center;
      height: 125px;
      color: #fff;
      background: linear-gradient(135deg, var(--accent), #d9b79a);
      font-size: 34px;
    }
    .body { padding: 16px; }
    .body h3 { margin: 0 0 5px; font-size: 17px; }
    .body p { margin: 3px 0; color: var(--muted); font-size: 13px; }
    .row {
      display: flex;
      align-items: center;
      justify-content: space-between;
      gap: 10px;
      margin-top: 14px;
    }
    .price { color: var(--accent); font-weight: 800; }
    .action { border-radius: 9px; padding: 9px 12px; }
    .notice { min-height: 24px; margin: 16px 2px; color: var(--accent); font-weight: 700; }
    footer {
      margin-top: 35px;
      padding: 18px 4px;
      border-top: 1px solid var(--line);
      color: var(--muted);
      font-size: 12px;
    }
    @media (max-width: 600px) {
      header { align-items: flex-start; flex-wrap: wrap; }
      nav { order: 3; width: 100%; gap: 12px; flex-wrap: wrap; }
      main { margin: 22px auto; }
      .hero { padding: 24px; }
    }
  </style>
</head>
<body>
  <header>
    <div class="brand">RAILWAY</div>
    <nav><span>Explore</span><span>Services</span><span>About</span></nav>
    <button class="cart" id="cartButton">My selections (0)</button>
  </header>
  <main>
    <section class="hero">
```

```
      <div class="eyebrow">Internet Programming · Experiment 04</div>
      <h1>Online Train Reservation</h1>
      <p>Plan your next rail journey</p>
    </section>
    <section class="search" aria-label="Search and filters">
      <input id="searchBox" type="search" placeholder="Search by name or detail" aria-label="Search">
      <select id="category">
        <option value="">All options</option>
        <option value="popular">Popular</option>
        <option value="available">Available now</option>
      </select>
      <button class="action" id="searchButton">Search</button>
    </section>
    <div class="sectionhead">
      <h2>Recommended for you</h2>
      <span id="countLabel">3 options</span>
    </div>
    <section class="grid" id="catalog" aria-live="polite"></section>
    <div class="notice" id="notice" role="status">Choose an option to continue.</div>
    <footer>
      Demonstration interface with sample data. Authentication, live availability, payments and
      notifications require a server integration.
    </footer>
  </main>
  <script>
    const items = [
      {
        name: "12675 · Kovai Express",
        line: "Chennai Central → Coimbatore",
        meta: "Chair Car · Available",
        price: "■680"
      },
      {
        name: "12635 · Vaigai Express",
        line: "Chennai Egmore → Madurai",
        meta: "2A · Available",
        price: "■1,145"
      },
      {
        name: "12658 · Chennai Mail",
        line: "Bengaluru → Chennai Central",
        meta: "3A · RAC 4",
        price: "■920"
      }
    ];
    let selected = 0;
    const catalog = document.querySelector('#catalog');
    const notice = document.querySelector('#notice');

    function render(list = items) {
      catalog.innerHTML = '';
      document.querySelector('#countLabel').textContent = `${list.length} options`;
      list.forEach((item, index) => {
        const card = document.createElement('article');
        card.className = 'card';
        card.innerHTML = `
          <div class="art" aria-hidden="true">${['❖', '■', '✧'][index % 3]}</div>
          <div class="body">
            <h3>${item.name}</h3>
            <p>${item.line}</p>
            <p>${item.meta}</p>
            <div class="row">
              <span class="price">${item.price}</span>
              <button class="action" data-index="${index}">Select</button>
            </div>
          </div>`;
        catalog.append(card);
      });
    }

    document.querySelector('#searchButton').addEventListener('click', () => {
      const query = document.querySelector('#searchBox').value.trim().toLowerCase();
      const matches = items.filter(item =>
        `${item.name} ${item.line} ${item.meta}`.toLowerCase().includes(query)
      );
      render(matches);
      notice.textContent = query ? 'Search results updated.' : 'Showing all options.';
    });

    catalog.addEventListener('click', event => {
      const button = event.target.closest('button[data-index]');
      if (!button) return;
      const chosen = catalog.querySelectorAll('button[data-index]')[Number(button.dataset.index)];
      selected += 1;
      document.querySelector('#cartButton').textContent = `My selections (${selected})`;
      notice.textContent = `${chosen.closest('.body').querySelector('h3').textContent} added to your
selections.`;
    });

    document.querySelector('#cartButton').addEventListener('click', () => {
      notice.textContent = selected
        ? `You have ${selected} item(s) selected. Review your request.`
        : 'Your selections are empty. Choose an option first.';
    });

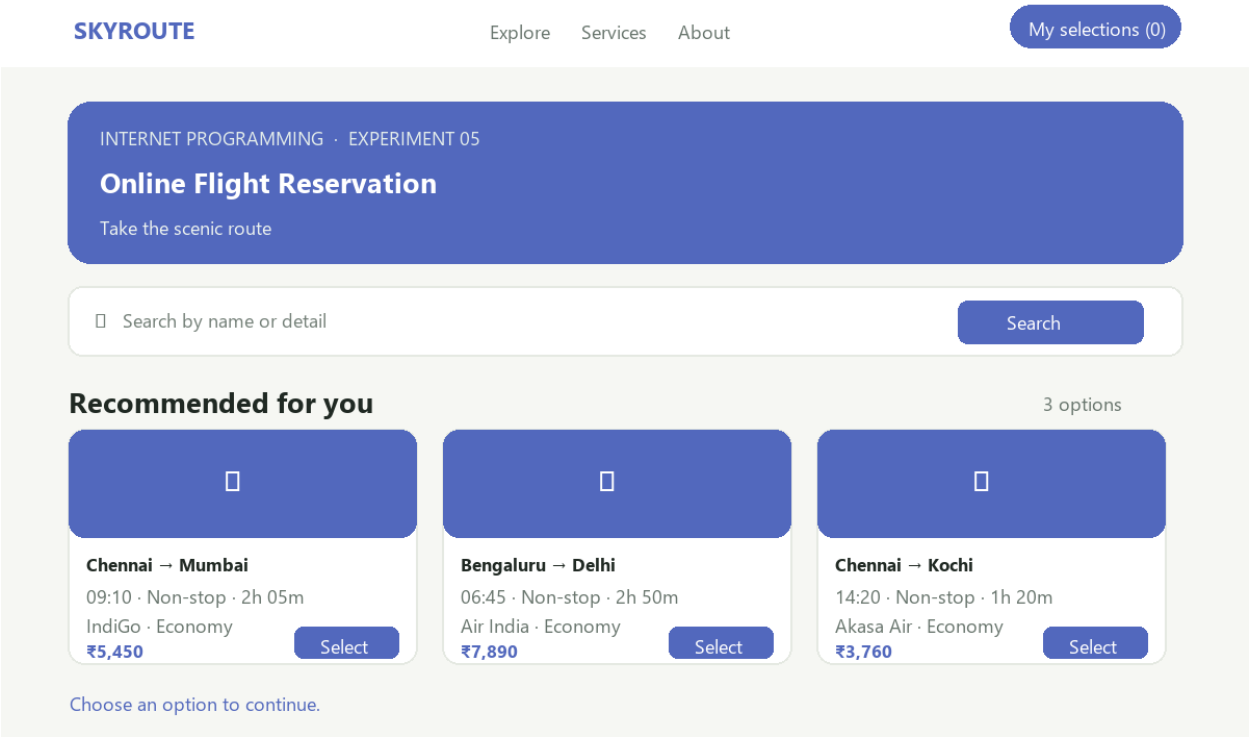
    render();
  </script>
```

```
</body>
</html>
```

Experiment 5: Online Flight Reservation

Aim: To implement a flight search and booking front end that lists available routes, departure times, carriers and sample fares, with a user action to select a flight.

Output screenshot



Result: The sample flight routes, carriers, timings and fares were displayed, and the user can select a flight.

Complete source code (standalone HTML)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Online Flight Reservation</title>
  <style>
    :root {
      --ink: #202923;
      --muted: #68756d;
      --paper: #f6f7f3;
      --card: #fff;
      --line: #e5e9e2;
      --accent: #5268bd;
    }
    * { box-sizing: border-box; }
    body {
      margin: 0;
      background: var(--paper);
      color: var(--ink);
      font: 15px/1.5 system-ui, -apple-system, "Segoe UI", sans-serif;
    }
    header {
      display: flex;
      align-items: center;
      justify-content: space-between;
      gap: 20px;
      padding: 16px max(5vw, 20px);
      background: #fff;
      border-bottom: 1px solid var(--line);
    }
    .brand { color: var(--accent); font-size: 18px; font-weight: 800; letter-spacing: .12em; }
    nav { display: flex; gap: 22px; color: var(--muted); font-size: 14px; }
    .cart, .action {
```

```
      border: 0;
      background: var(--accent);
      color: #fff;
      font-weight: 700;
      cursor: pointer;
    }
    .cart { border-radius: 99px; padding: 10px 16px; }
    main { max-width: 1100px; margin: 40px auto; padding: 0 22px; }
    .hero {
      padding: 30px 32px;
      border-radius: 22px;
      color: #fff;
      background: linear-gradient(120deg, var(--accent), #273744);
    }
    .eyebrow { text-transform: uppercase; letter-spacing: .14em; font-size: 11px; opacity: .8; }
    h1 { margin: 7px 0 8px; font-size: clamp(28px, 4vw, 42px); line-height: 1.1; }
    .hero p { margin: 0; opacity: .88; }
    .search {
      display: flex;
      flex-wrap: wrap;
      gap: 10px;
      margin: 20px 0 24px;
      padding: 14px;
      border: 1px solid var(--line);
      border-radius: 16px;
      background: #fff;
    }
    input, select {
      min-width: 150px;
      flex: 1;
      padding: 11px 12px;
      border: 1px solid var(--line);
      border-radius: 10px;
      color: var(--ink);
      background: #fff;
      font: inherit;
    }
    .sectionhead {
      display: flex;
      align-items: end;
      justify-content: space-between;
      margin: 26px 0 14px;
    }
    h2 { margin: 0; font-size: 22px; }
    .sectionhead span { color: var(--muted); font-size: 13px; }
    .grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(245px, 1fr)); gap: 16px; }
    .card {
      overflow: hidden;
      border: 1px solid var(--line);
      border-radius: 16px;
      background: var(--card);
      box-shadow: 0 5px 18px #2233220a;
    }
    .art {
      display: grid;
      place-items: center;
      height: 125px;
      color: #fff;
      background: linear-gradient(135deg, var(--accent), #d9b79a);
      font-size: 34px;
    }
    .body { padding: 16px; }
    .body h3 { margin: 0 0 5px; font-size: 17px; }
    .body p { margin: 3px 0; color: var(--muted); font-size: 13px; }
    .row {
      display: flex;
      align-items: center;
      justify-content: space-between;
      gap: 10px;
      margin-top: 14px;
    }
    .price { color: var(--accent); font-weight: 800; }
    .action { border-radius: 9px; padding: 9px 12px; }
    .notice { min-height: 24px; margin: 16px 2px; color: var(--accent); font-weight: 700; }
    footer {
      margin-top: 35px;
      padding: 18px 4px;
      border-top: 1px solid var(--line);
      color: var(--muted);
      font-size: 12px;
    }
    @media (max-width: 600px) {
      header { align-items: flex-start; flex-wrap: wrap; }
      nav { order: 3; width: 100%; gap: 12px; flex-wrap: wrap; }
      main { margin: 22px auto; }
      .hero { padding: 24px; }
    }
  </style>
</head>
<body>
  <header>
    <div class="brand">SKYROUTE</div>
    <nav><span>Explore</span><span>Services</span><span>About</span></nav>
    <button class="cart" id="cartButton">My selections (0)</button>
  </header>
  <main>
    <section class="hero">
```

```
      <div class="eyebrow">Internet Programming · Experiment 05</div>
      <h1>Online Flight Reservation</h1>
      <p>Take the scenic route</p>
    </section>
    <section class="search" aria-label="Search and filters">
      <input id="searchBox" type="search" placeholder="Search by name or detail" aria-label="Search">
      <select id="category">
        <option value="">All options</option>
        <option value="popular">Popular</option>
        <option value="available">Available now</option>
      </select>
      <button class="action" id="searchButton">Search</button>
    </section>
    <div class="sectionhead">
      <h2>Recommended for you</h2>
      <span id="countLabel">3 options</span>
    </div>
    <section class="grid" id="catalog" aria-live="polite"></section>
    <div class="notice" id="notice" role="status">Choose an option to continue.</div>
    <footer>
      Demonstration interface with sample data. Authentication, live availability, payments and
      notifications require a server integration.
    </footer>
  </main>
  <script>
    const items = [
      {
        name: "Chennai → Mumbai",
        line: "09:10 · Non-stop · 2h 05m",
        meta: "IndiGo · Economy",
        price: "■5,450"
      },
      {
        name: "Bengaluru → Delhi",
        line: "06:45 · Non-stop · 2h 50m",
        meta: "Air India · Economy",
        price: "■7,890"
      },
      {
        name: "Chennai → Kochi",
        line: "14:20 · Non-stop · 1h 20m",
        meta: "Akasa Air · Economy",
        price: "■3,760"
      }
    ];
    let selected = 0;
    const catalog = document.querySelector('#catalog');
    const notice = document.querySelector('#notice');

    function render(list = items) {
      catalog.innerHTML = '';
      document.querySelector('#countLabel').textContent = `${list.length} options`;
      list.forEach((item, index) => {
        const card = document.createElement('article');
        card.className = 'card';
        card.innerHTML = `
          <div class="art" aria-hidden="true">${['❖', '■', '✧'][index % 3]}</div>
          <div class="body">
            <h3>${item.name}</h3>
            <p>${item.line}</p>
            <p>${item.meta}</p>
            <div class="row">
              <span class="price">${item.price}</span>
              <button class="action" data-index="${index}">Select</button>
            </div>
          </div>`;
        catalog.append(card);
      });
    }

    document.querySelector('#searchButton').addEventListener('click', () => {
      const query = document.querySelector('#searchBox').value.trim().toLowerCase();
      const matches = items.filter(item =>
        `${item.name} ${item.line} ${item.meta}`.toLowerCase().includes(query)
      );
      render(matches);
      notice.textContent = query ? 'Search results updated.' : 'Showing all options.';
    });

    catalog.addEventListener('click', event => {
      const button = event.target.closest('button[data-index]');
      if (!button) return;
      const chosen = catalog.querySelectorAll('button[data-index]')[Number(button.dataset.index)];
      selected += 1;
      document.querySelector('#cartButton').textContent = `My selections (${selected})`;
      notice.textContent = `${chosen.closest('.body').querySelector('h3').textContent} added to your
selections.`;
    });

    document.querySelector('#cartButton').addEventListener('click', () => {
      notice.textContent = selected
        ? `You have ${selected} item(s) selected. Review your request.`
        : 'Your selections are empty. Choose an option first.';
    });

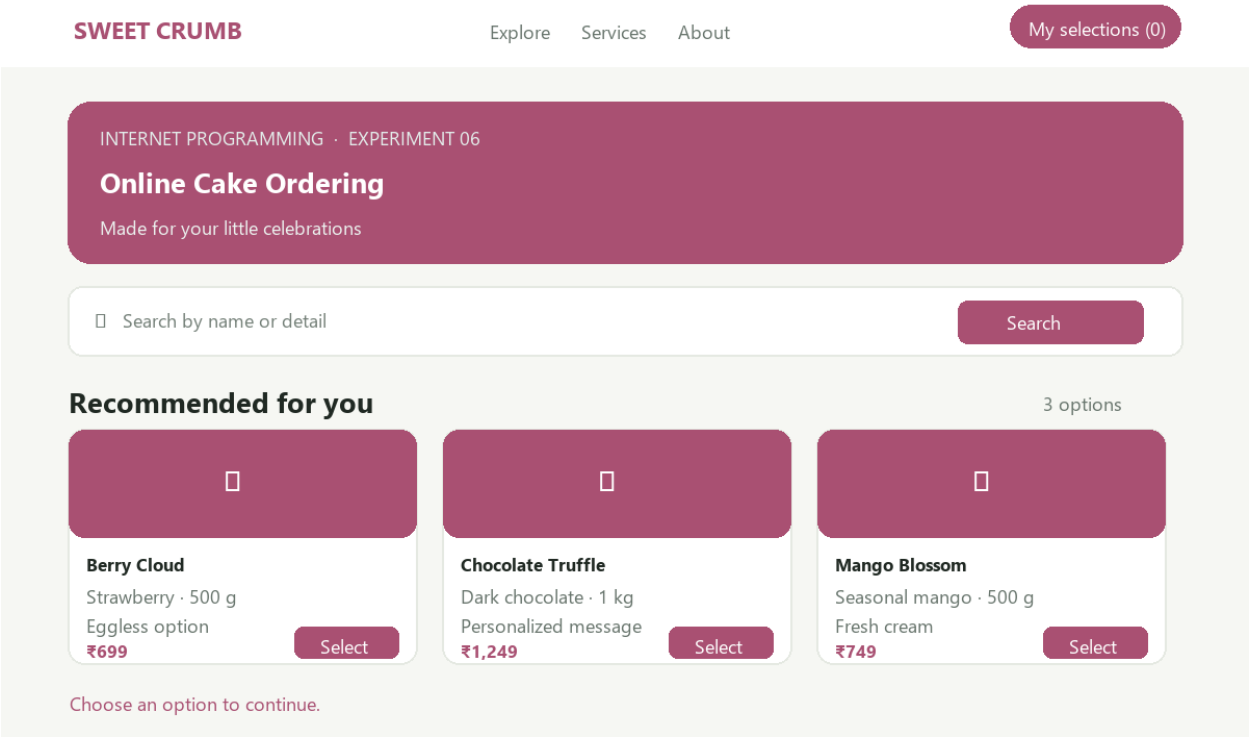
    render();
  </script>
```

```
</body>
</html>
```

Experiment 6: Online Cake Ordering

Aim: To create an online cake shop where customers can browse cakes, search the menu, add a selection to a cart, and see the cart total.

Output screenshot



Result: The cake catalogue and customisation details were displayed, and selecting a cake updates the demo cart.

Complete source code (standalone HTML)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Online Cake Ordering</title>
  <style>
    :root {
      --ink: #202923;
      --muted: #68756d;
      --paper: #f6f7f3;
      --card: #fff;
      --line: #e5e9e2;
      --accent: #a95072;
    }
    * { box-sizing: border-box; }
    body {
      margin: 0;
      background: var(--paper);
      color: var(--ink);
      font: 15px/1.5 system-ui, -apple-system, "Segoe UI", sans-serif;
    }
    header {
      display: flex;
      align-items: center;
      justify-content: space-between;
      gap: 20px;
      padding: 16px max(5vw, 20px);
      background: #fff;
      border-bottom: 1px solid var(--line);
    }
    .brand { color: var(--accent); font-size: 18px; font-weight: 800; letter-spacing: .12em; }
    nav { display: flex; gap: 22px; color: var(--muted); font-size: 14px; }
    .cart, .action {
```

```
      border: 0;
      background: var(--accent);
      color: #fff;
      font-weight: 700;
      cursor: pointer;
    }
    .cart { border-radius: 99px; padding: 10px 16px; }
    main { max-width: 1100px; margin: 40px auto; padding: 0 22px; }
    .hero {
      padding: 30px 32px;
      border-radius: 22px;
      color: #fff;
      background: linear-gradient(120deg, var(--accent), #273744);
    }
    .eyebrow { text-transform: uppercase; letter-spacing: .14em; font-size: 11px; opacity: .8; }
    h1 { margin: 7px 0 8px; font-size: clamp(28px, 4vw, 42px); line-height: 1.1; }
    .hero p { margin: 0; opacity: .88; }
    .search {
      display: flex;
      flex-wrap: wrap;
      gap: 10px;
      margin: 20px 0 24px;
      padding: 14px;
      border: 1px solid var(--line);
      border-radius: 16px;
      background: #fff;
    }
    input, select {
      min-width: 150px;
      flex: 1;
      padding: 11px 12px;
      border: 1px solid var(--line);
      border-radius: 10px;
      color: var(--ink);
      background: #fff;
      font: inherit;
    }
    .sectionhead {
      display: flex;
      align-items: end;
      justify-content: space-between;
      margin: 26px 0 14px;
    }
    h2 { margin: 0; font-size: 22px; }
    .sectionhead span { color: var(--muted); font-size: 13px; }
    .grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(245px, 1fr)); gap: 16px; }
    .card {
      overflow: hidden;
      border: 1px solid var(--line);
      border-radius: 16px;
      background: var(--card);
      box-shadow: 0 5px 18px #2233220a;
    }
    .art {
      display: grid;
      place-items: center;
      height: 125px;
      color: #fff;
      background: linear-gradient(135deg, var(--accent), #d9b79a);
      font-size: 34px;
    }
    .body { padding: 16px; }
    .body h3 { margin: 0 0 5px; font-size: 17px; }
    .body p { margin: 3px 0; color: var(--muted); font-size: 13px; }
    .row {
      display: flex;
      align-items: center;
      justify-content: space-between;
      gap: 10px;
      margin-top: 14px;
    }
    .price { color: var(--accent); font-weight: 800; }
    .action { border-radius: 9px; padding: 9px 12px; }
    .notice { min-height: 24px; margin: 16px 2px; color: var(--accent); font-weight: 700; }
    footer {
      margin-top: 35px;
      padding: 18px 4px;
      border-top: 1px solid var(--line);
      color: var(--muted);
      font-size: 12px;
    }
    @media (max-width: 600px) {
      header { align-items: flex-start; flex-wrap: wrap; }
      nav { order: 3; width: 100%; gap: 12px; flex-wrap: wrap; }
      main { margin: 22px auto; }
      .hero { padding: 24px; }
    }
  </style>
</head>
<body>
  <header>
    <div class="brand">SWEET CRUMB</div>
    <nav><span>Explore</span><span>Services</span><span>About</span></nav>
    <button class="cart" id="cartButton">My selections (0)</button>
  </header>
  <main>
    <section class="hero">
```

```
      <div class="eyebrow">Internet Programming · Experiment 06</div>
      <h1>Online Cake Ordering</h1>
      <p>Made for your little celebrations</p>
    </section>
    <section class="search" aria-label="Search and filters">
      <input id="searchBox" type="search" placeholder="Search by name or detail" aria-label="Search">
      <select id="category">
        <option value="">All options</option>
        <option value="popular">Popular</option>
        <option value="available">Available now</option>
      </select>
      <button class="action" id="searchButton">Search</button>
    </section>
    <div class="sectionhead">
      <h2>Recommended for you</h2>
      <span id="countLabel">3 options</span>
    </div>
    <section class="grid" id="catalog" aria-live="polite"></section>
    <div class="notice" id="notice" role="status">Choose an option to continue.</div>
    <footer>
      Demonstration interface with sample data. Authentication, live availability, payments and
      notifications require a server integration.
    </footer>
  </main>
  <script>
    const items = [
      {
        name: "Berry Cloud",
        line: "Strawberry · 500 g",
        meta: "Eggless option",
        price: "■699"
      },
      {
        name: "Chocolate Truffle",
        line: "Dark chocolate · 1 kg",
        meta: "Personalized message",
        price: "■1,249"
      },
      {
        name: "Mango Blossom",
        line: "Seasonal mango · 500 g",
        meta: "Fresh cream",
        price: "■749"
      }
    ];
    let selected = 0;
    const catalog = document.querySelector('#catalog');
    const notice = document.querySelector('#notice');

    function render(list = items) {
      catalog.innerHTML = '';
      document.querySelector('#countLabel').textContent = `${list.length} options`;
      list.forEach((item, index) => {
        const card = document.createElement('article');
        card.className = 'card';
        card.innerHTML = `
          <div class="art" aria-hidden="true">${['❖', '■', '✧'][index % 3]}</div>
          <div class="body">
            <h3>${item.name}</h3>
            <p>${item.line}</p>
            <p>${item.meta}</p>
            <div class="row">
              <span class="price">${item.price}</span>
              <button class="action" data-index="${index}">Select</button>
            </div>
          </div>`;
        catalog.append(card);
      });
    }

    document.querySelector('#searchButton').addEventListener('click', () => {
      const query = document.querySelector('#searchBox').value.trim().toLowerCase();
      const matches = items.filter(item =>
        `${item.name} ${item.line} ${item.meta}`.toLowerCase().includes(query)
      );
      render(matches);
      notice.textContent = query ? 'Search results updated.' : 'Showing all options.';
    });

    catalog.addEventListener('click', event => {
      const button = event.target.closest('button[data-index]');
      if (!button) return;
      const chosen = catalog.querySelectorAll('button[data-index]')[Number(button.dataset.index)];
      selected += 1;
      document.querySelector('#cartButton').textContent = `My selections (${selected})`;
      notice.textContent = `${chosen.closest('.body').querySelector('h3').textContent} added to your
selections.`;
    });

    document.querySelector('#cartButton').addEventListener('click', () => {
      notice.textContent = selected
        ? `You have ${selected} item(s) selected. Review your request.`
        : 'Your selections are empty. Choose an option first.';
    });

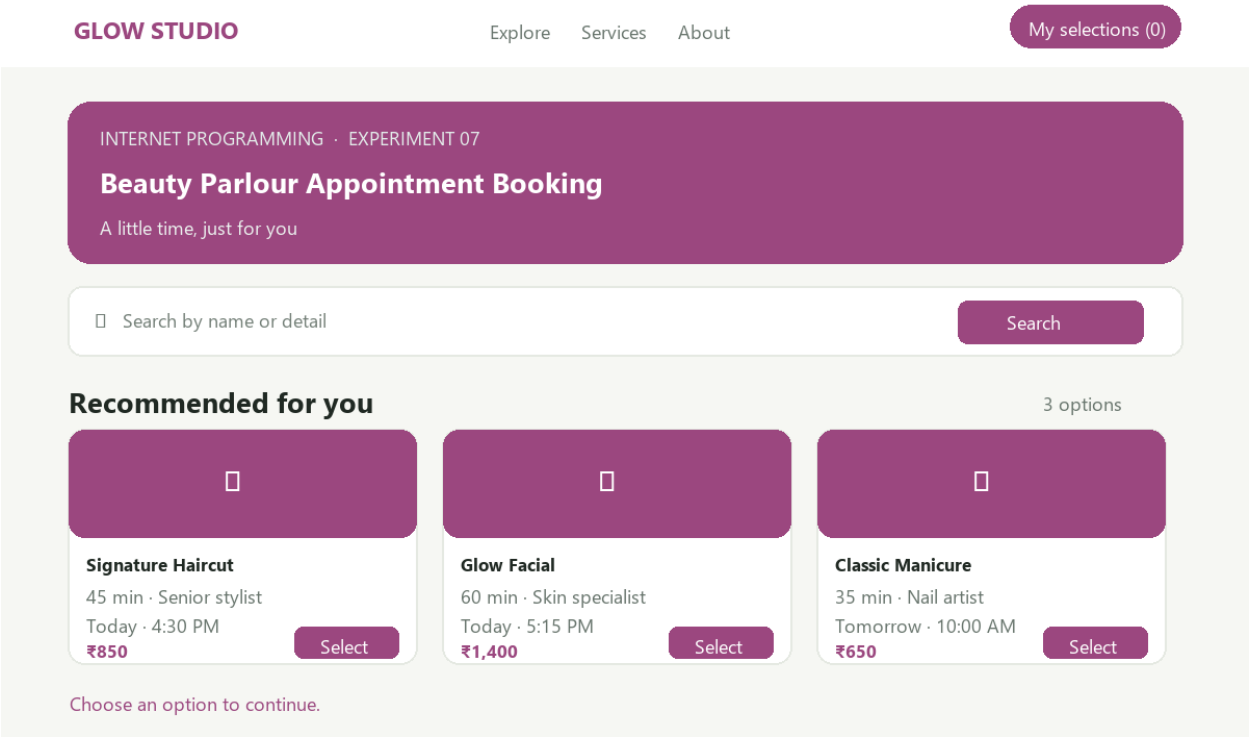
    render();
  </script>
```

```
</body>
</html>
```

Experiment 7: Beauty Parlour Appointment Booking

Aim: To build a salon appointment page that presents services, duration, price and sample time slots, and lets a customer request an appointment.

Output screenshot



Result: The salon services, sample stylist slots and prices were displayed, and a customer can select a service.

Complete source code (standalone HTML)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Beauty Parlour Appointment Booking</title>
  <style>
    :root {
      --ink: #202923;
      --muted: #68756d;
      --paper: #f6f7f3;
      --card: #fff;
      --line: #e5e9e2;
      --accent: #9b477f;
    }
    * { box-sizing: border-box; }
    body {
      margin: 0;
      background: var(--paper);
      color: var(--ink);
      font: 15px/1.5 system-ui, -apple-system, "Segoe UI", sans-serif;
    }
    header {
      display: flex;
      align-items: center;
      justify-content: space-between;
      gap: 20px;
      padding: 16px max(5vw, 20px);
      background: #fff;
      border-bottom: 1px solid var(--line);
    }
    .brand { color: var(--accent); font-size: 18px; font-weight: 800; letter-spacing: .12em; }
    nav { display: flex; gap: 22px; color: var(--muted); font-size: 14px; }
    .cart, .action {
```

```
      border: 0;
      background: var(--accent);
      color: #fff;
      font-weight: 700;
      cursor: pointer;
    }
    .cart { border-radius: 99px; padding: 10px 16px; }
    main { max-width: 1100px; margin: 40px auto; padding: 0 22px; }
    .hero {
      padding: 30px 32px;
      border-radius: 22px;
      color: #fff;
      background: linear-gradient(120deg, var(--accent), #273744);
    }
    .eyebrow { text-transform: uppercase; letter-spacing: .14em; font-size: 11px; opacity: .8; }
    h1 { margin: 7px 0 8px; font-size: clamp(28px, 4vw, 42px); line-height: 1.1; }
    .hero p { margin: 0; opacity: .88; }
    .search {
      display: flex;
      flex-wrap: wrap;
      gap: 10px;
      margin: 20px 0 24px;
      padding: 14px;
      border: 1px solid var(--line);
      border-radius: 16px;
      background: #fff;
    }
    input, select {
      min-width: 150px;
      flex: 1;
      padding: 11px 12px;
      border: 1px solid var(--line);
      border-radius: 10px;
      color: var(--ink);
      background: #fff;
      font: inherit;
    }
    .sectionhead {
      display: flex;
      align-items: end;
      justify-content: space-between;
      margin: 26px 0 14px;
    }
    h2 { margin: 0; font-size: 22px; }
    .sectionhead span { color: var(--muted); font-size: 13px; }
    .grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(245px, 1fr)); gap: 16px; }
    .card {
      overflow: hidden;
      border: 1px solid var(--line);
      border-radius: 16px;
      background: var(--card);
      box-shadow: 0 5px 18px #2233220a;
    }
    .art {
      display: grid;
      place-items: center;
      height: 125px;
      color: #fff;
      background: linear-gradient(135deg, var(--accent), #d9b79a);
      font-size: 34px;
    }
    .body { padding: 16px; }
    .body h3 { margin: 0 0 5px; font-size: 17px; }
    .body p { margin: 3px 0; color: var(--muted); font-size: 13px; }
    .row {
      display: flex;
      align-items: center;
      justify-content: space-between;
      gap: 10px;
      margin-top: 14px;
    }
    .price { color: var(--accent); font-weight: 800; }
    .action { border-radius: 9px; padding: 9px 12px; }
    .notice { min-height: 24px; margin: 16px 2px; color: var(--accent); font-weight: 700; }
    footer {
      margin-top: 35px;
      padding: 18px 4px;
      border-top: 1px solid var(--line);
      color: var(--muted);
      font-size: 12px;
    }
    @media (max-width: 600px) {
      header { align-items: flex-start; flex-wrap: wrap; }
      nav { order: 3; width: 100%; gap: 12px; flex-wrap: wrap; }
      main { margin: 22px auto; }
      .hero { padding: 24px; }
    }
  </style>
</head>
<body>
  <header>
    <div class="brand">GLOW STUDIO</div>
    <nav><span>Explore</span><span>Services</span><span>About</span></nav>
    <button class="cart" id="cartButton">My selections (0)</button>
  </header>
  <main>
    <section class="hero">
```

```
      <div class="eyebrow">Internet Programming · Experiment 07</div>
      <h1>Beauty Parlour Appointment Booking</h1>
      <p>A little time, just for you</p>
    </section>
    <section class="search" aria-label="Search and filters">
      <input id="searchBox" type="search" placeholder="Search by name or detail" aria-label="Search">
      <select id="category">
        <option value="">All options</option>
        <option value="popular">Popular</option>
        <option value="available">Available now</option>
      </select>
      <button class="action" id="searchButton">Search</button>
    </section>
    <div class="sectionhead">
      <h2>Recommended for you</h2>
      <span id="countLabel">3 options</span>
    </div>
    <section class="grid" id="catalog" aria-live="polite"></section>
    <div class="notice" id="notice" role="status">Choose an option to continue.</div>
    <footer>
      Demonstration interface with sample data. Authentication, live availability, payments and
      notifications require a server integration.
    </footer>
  </main>
  <script>
    const items = [
      {
        name: "Signature Haircut",
        line: "45 min · Senior stylist",
        meta: "Today · 4:30 PM",
        price: "■850"
      },
      {
        name: "Glow Facial",
        line: "60 min · Skin specialist",
        meta: "Today · 5:15 PM",
        price: "■1,400"
      },
      {
        name: "Classic Manicure",
        line: "35 min · Nail artist",
        meta: "Tomorrow · 10:00 AM",
        price: "■650"
      }
    ];
    let selected = 0;
    const catalog = document.querySelector('#catalog');
    const notice = document.querySelector('#notice');

    function render(list = items) {
      catalog.innerHTML = '';
      document.querySelector('#countLabel').textContent = `${list.length} options`;
      list.forEach((item, index) => {
        const card = document.createElement('article');
        card.className = 'card';
        card.innerHTML = `
          <div class="art" aria-hidden="true">${['❖', '■', '✧'][index % 3]}</div>
          <div class="body">
            <h3>${item.name}</h3>
            <p>${item.line}</p>
            <p>${item.meta}</p>
            <div class="row">
              <span class="price">${item.price}</span>
              <button class="action" data-index="${index}">Select</button>
            </div>
          </div>`;
        catalog.append(card);
      });
    }

    document.querySelector('#searchButton').addEventListener('click', () => {
      const query = document.querySelector('#searchBox').value.trim().toLowerCase();
      const matches = items.filter(item =>
        `${item.name} ${item.line} ${item.meta}`.toLowerCase().includes(query)
      );
      render(matches);
      notice.textContent = query ? 'Search results updated.' : 'Showing all options.';
    });

    catalog.addEventListener('click', event => {
      const button = event.target.closest('button[data-index]');
      if (!button) return;
      const chosen = catalog.querySelectorAll('button[data-index]')[Number(button.dataset.index)];
      selected += 1;
      document.querySelector('#cartButton').textContent = `My selections (${selected})`;
      notice.textContent = `${chosen.closest('.body').querySelector('h3').textContent} added to your
selections.`;
    });

    document.querySelector('#cartButton').addEventListener('click', () => {
      notice.textContent = selected
        ? `You have ${selected} item(s) selected. Review your request.`
        : 'Your selections are empty. Choose an option first.';
    });

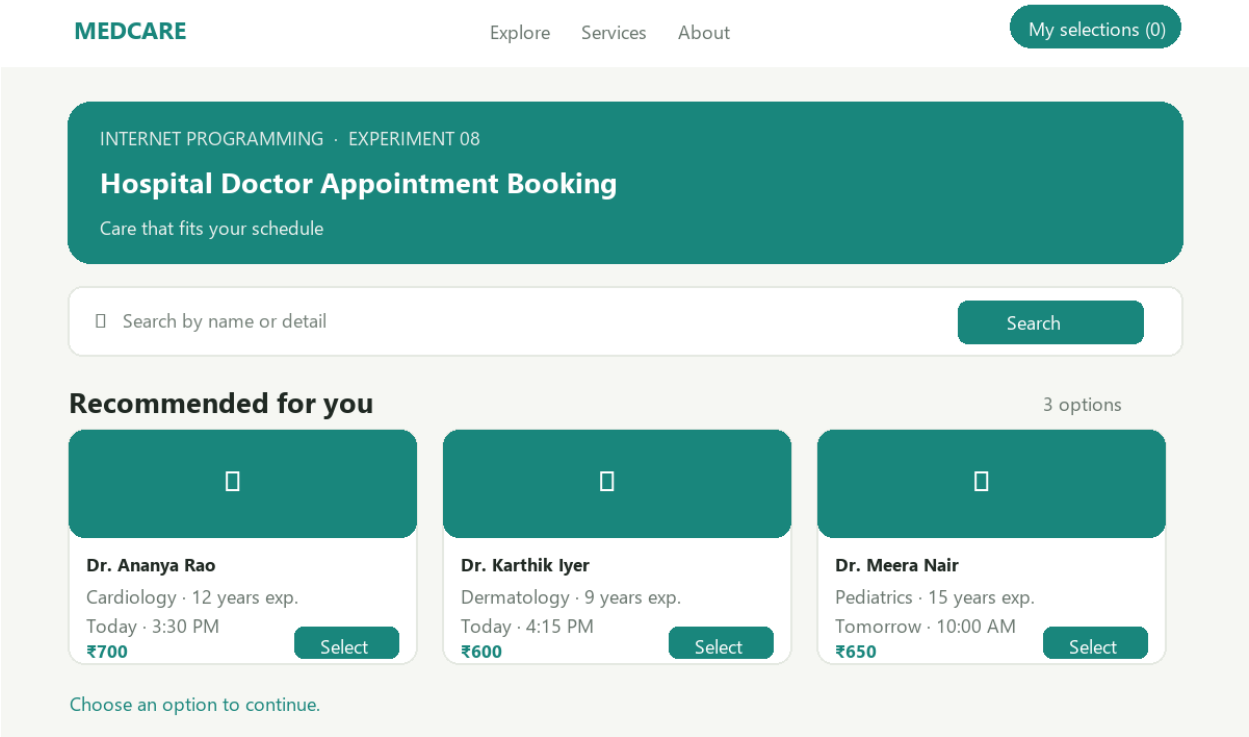
    render();
  </script>
```

```
</body>
</html>
```

Experiment 8: Hospital Doctor Appointment Booking

Aim: To design a patient-facing doctor directory with specialty search, appointment slots and a sample booking confirmation. This front-end demonstration uses fictional providers and data.

Output screenshot



Result: The fictional doctor directory and appointment slots were displayed, and the patient can select a provider to request an appointment.

Complete source code (standalone HTML)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Hospital Doctor Appointment Booking</title>
  <style>
    :root {
      --ink: #202923;
      --muted: #68756d;
      --paper: #f6f7f3;
      --card: #fff;
      --line: #e5e9e2;
      --accent: #19867c;
    }
    * { box-sizing: border-box; }
    body {
      margin: 0;
      background: var(--paper);
      color: var(--ink);
      font: 15px/1.5 system-ui, -apple-system, "Segoe UI", sans-serif;
    }
    header {
      display: flex;
      align-items: center;
      justify-content: space-between;
      gap: 20px;
      padding: 16px max(5vw, 20px);
      background: #fff;
      border-bottom: 1px solid var(--line);
    }
    .brand { color: var(--accent); font-size: 18px; font-weight: 800; letter-spacing: .12em; }
    nav { display: flex; gap: 22px; color: var(--muted); font-size: 14px; }
    .cart, .action {
```

```
      border: 0;
      background: var(--accent);
      color: #fff;
      font-weight: 700;
      cursor: pointer;
    }
    .cart { border-radius: 99px; padding: 10px 16px; }
    main { max-width: 1100px; margin: 40px auto; padding: 0 22px; }
    .hero {
      padding: 30px 32px;
      border-radius: 22px;
      color: #fff;
      background: linear-gradient(120deg, var(--accent), #273744);
    }
    .eyebrow { text-transform: uppercase; letter-spacing: .14em; font-size: 11px; opacity: .8; }
    h1 { margin: 7px 0 8px; font-size: clamp(28px, 4vw, 42px); line-height: 1.1; }
    .hero p { margin: 0; opacity: .88; }
    .search {
      display: flex;
      flex-wrap: wrap;
      gap: 10px;
      margin: 20px 0 24px;
      padding: 14px;
      border: 1px solid var(--line);
      border-radius: 16px;
      background: #fff;
    }
    input, select {
      min-width: 150px;
      flex: 1;
      padding: 11px 12px;
      border: 1px solid var(--line);
      border-radius: 10px;
      color: var(--ink);
      background: #fff;
      font: inherit;
    }
    .sectionhead {
      display: flex;
      align-items: end;
      justify-content: space-between;
      margin: 26px 0 14px;
    }
    h2 { margin: 0; font-size: 22px; }
    .sectionhead span { color: var(--muted); font-size: 13px; }
    .grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(245px, 1fr)); gap: 16px; }
    .card {
      overflow: hidden;
      border: 1px solid var(--line);
      border-radius: 16px;
      background: var(--card);
      box-shadow: 0 5px 18px #2233220a;
    }
    .art {
      display: grid;
      place-items: center;
      height: 125px;
      color: #fff;
      background: linear-gradient(135deg, var(--accent), #d9b79a);
      font-size: 34px;
    }
    .body { padding: 16px; }
    .body h3 { margin: 0 0 5px; font-size: 17px; }
    .body p { margin: 3px 0; color: var(--muted); font-size: 13px; }
    .row {
      display: flex;
      align-items: center;
      justify-content: space-between;
      gap: 10px;
      margin-top: 14px;
    }
    .price { color: var(--accent); font-weight: 800; }
    .action { border-radius: 9px; padding: 9px 12px; }
    .notice { min-height: 24px; margin: 16px 2px; color: var(--accent); font-weight: 700; }
    footer {
      margin-top: 35px;
      padding: 18px 4px;
      border-top: 1px solid var(--line);
      color: var(--muted);
      font-size: 12px;
    }
    @media (max-width: 600px) {
      header { align-items: flex-start; flex-wrap: wrap; }
      nav { order: 3; width: 100%; gap: 12px; flex-wrap: wrap; }
      main { margin: 22px auto; }
      .hero { padding: 24px; }
    }
  </style>
</head>
<body>
  <header>
    <div class="brand">MEDCARE</div>
    <nav><span>Explore</span><span>Services</span><span>About</span></nav>
    <button class="cart" id="cartButton">My selections (0)</button>
  </header>
  <main>
    <section class="hero">
```

```
      <div class="eyebrow">Internet Programming · Experiment 08</div>
      <h1>Hospital Doctor Appointment Booking</h1>
      <p>Care that fits your schedule</p>
    </section>
    <section class="search" aria-label="Search and filters">
      <input id="searchBox" type="search" placeholder="Search by name or detail" aria-label="Search">
      <select id="category">
        <option value="">All options</option>
        <option value="popular">Popular</option>
        <option value="available">Available now</option>
      </select>
      <button class="action" id="searchButton">Search</button>
    </section>
    <div class="sectionhead">
      <h2>Recommended for you</h2>
      <span id="countLabel">3 options</span>
    </div>
    <section class="grid" id="catalog" aria-live="polite"></section>
    <div class="notice" id="notice" role="status">Choose an option to continue.</div>
    <footer>
      Demonstration interface with sample data. Authentication, live availability, payments and
      notifications require a server integration.
    </footer>
  </main>
  <script>
    const items = [
      {
        name: "Dr. Ananya Rao",
        line: "Cardiology · 12 years exp.",
        meta: "Today · 3:30 PM",
        price: "■700"
      },
      {
        name: "Dr. Karthik Iyer",
        line: "Dermatology · 9 years exp.",
        meta: "Today · 4:15 PM",
        price: "■600"
      },
      {
        name: "Dr. Meera Nair",
        line: "Pediatrics · 15 years exp.",
        meta: "Tomorrow · 10:00 AM",
        price: "■650"
      }
    ];
    let selected = 0;
    const catalog = document.querySelector('#catalog');
    const notice = document.querySelector('#notice');

    function render(list = items) {
      catalog.innerHTML = '';
      document.querySelector('#countLabel').textContent = `${list.length} options`;
      list.forEach((item, index) => {
        const card = document.createElement('article');
        card.className = 'card';
        card.innerHTML = `
          <div class="art" aria-hidden="true">${['❖', '■', '✧'][index % 3]}</div>
          <div class="body">
            <h3>${item.name}</h3>
            <p>${item.line}</p>
            <p>${item.meta}</p>
            <div class="row">
              <span class="price">${item.price}</span>
              <button class="action" data-index="${index}">Select</button>
            </div>
          </div>`;
        catalog.append(card);
      });
    }

    document.querySelector('#searchButton').addEventListener('click', () => {
      const query = document.querySelector('#searchBox').value.trim().toLowerCase();
      const matches = items.filter(item =>
        `${item.name} ${item.line} ${item.meta}`.toLowerCase().includes(query)
      );
      render(matches);
      notice.textContent = query ? 'Search results updated.' : 'Showing all options.';
    });

    catalog.addEventListener('click', event => {
      const button = event.target.closest('button[data-index]');
      if (!button) return;
      const chosen = catalog.querySelectorAll('button[data-index]')[Number(button.dataset.index)];
      selected += 1;
      document.querySelector('#cartButton').textContent = `My selections (${selected})`;
      notice.textContent = `${chosen.closest('.body').querySelector('h3').textContent} added to your
selections.`;
    });

    document.querySelector('#cartButton').addEventListener('click', () => {
      notice.textContent = selected
        ? `You have ${selected} item(s) selected. Review your request.`
        : 'Your selections are empty. Choose an option first.';
    });

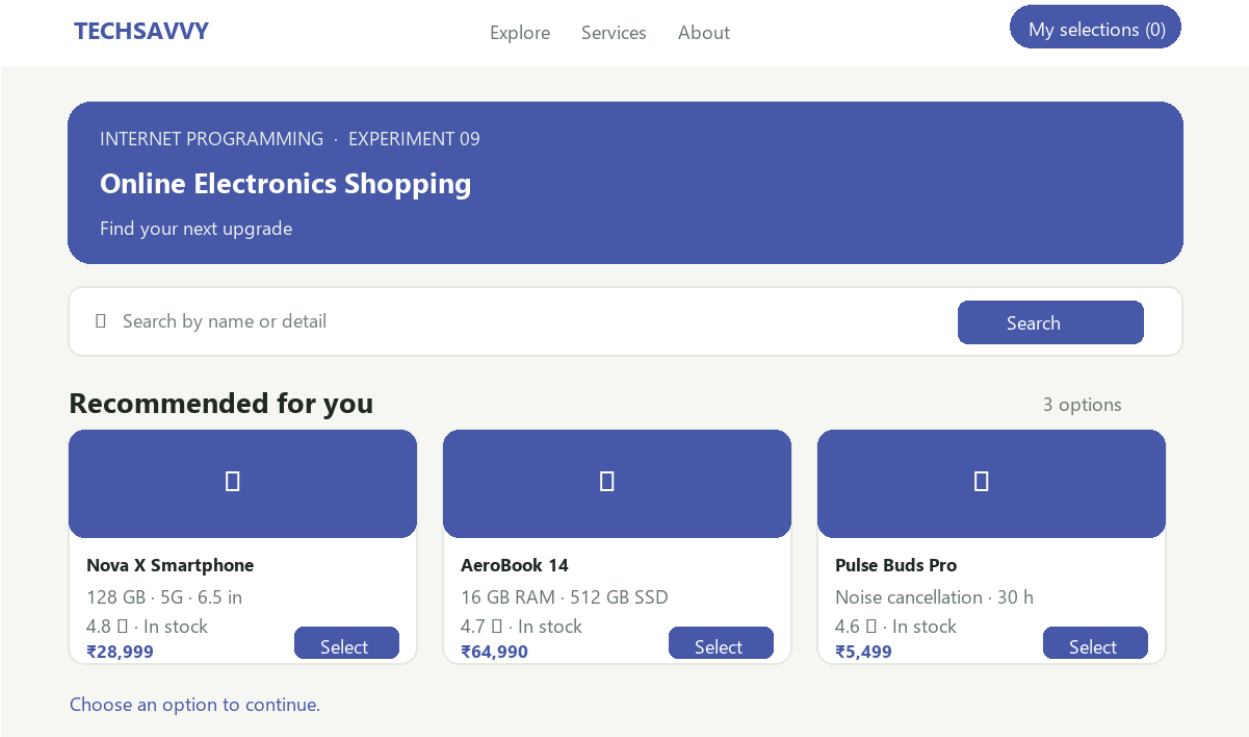
    render();
  </script>
```

```
</body>
</html>
```

Experiment 9: Online Electronics Shopping

Aim: To develop an electronics storefront with searchable products, category filters and a working sample cart that updates the selected-item count and subtotal.

Output screenshot



Result: The sample electronics catalogue was displayed, and search and selection controls update the visible results and cart count.

Complete source code (standalone HTML)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Online Electronics Shopping</title>
  <style>
    :root {
      --ink: #202923;
      --muted: #68756d;
      --paper: #f6f7f3;
      --card: #fff;
      --line: #e5e9e2;
      --accent: #4658a7;
    }
    * { box-sizing: border-box; }
    body {
      margin: 0;
      background: var(--paper);
      color: var(--ink);
      font: 15px/1.5 system-ui, -apple-system, "Segoe UI", sans-serif;
    }
    header {
      display: flex;
      align-items: center;
      justify-content: space-between;
      gap: 20px;
      padding: 16px max(5vw, 20px);
      background: #fff;
      border-bottom: 1px solid var(--line);
    }
    .brand { color: var(--accent); font-size: 18px; font-weight: 800; letter-spacing: .12em; }
    nav { display: flex; gap: 22px; color: var(--muted); font-size: 14px; }
    .cart, .action {
```

```
      border: 0;
      background: var(--accent);
      color: #fff;
      font-weight: 700;
      cursor: pointer;
    }
    .cart { border-radius: 99px; padding: 10px 16px; }
    main { max-width: 1100px; margin: 40px auto; padding: 0 22px; }
    .hero {
      padding: 30px 32px;
      border-radius: 22px;
      color: #fff;
      background: linear-gradient(120deg, var(--accent), #273744);
    }
    .eyebrow { text-transform: uppercase; letter-spacing: .14em; font-size: 11px; opacity: .8; }
    h1 { margin: 7px 0 8px; font-size: clamp(28px, 4vw, 42px); line-height: 1.1; }
    .hero p { margin: 0; opacity: .88; }
    .search {
      display: flex;
      flex-wrap: wrap;
      gap: 10px;
      margin: 20px 0 24px;
      padding: 14px;
      border: 1px solid var(--line);
      border-radius: 16px;
      background: #fff;
    }
    input, select {
      min-width: 150px;
      flex: 1;
      padding: 11px 12px;
      border: 1px solid var(--line);
      border-radius: 10px;
      color: var(--ink);
      background: #fff;
      font: inherit;
    }
    .sectionhead {
      display: flex;
      align-items: end;
      justify-content: space-between;
      margin: 26px 0 14px;
    }
    h2 { margin: 0; font-size: 22px; }
    .sectionhead span { color: var(--muted); font-size: 13px; }
    .grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(245px, 1fr)); gap: 16px; }
    .card {
      overflow: hidden;
      border: 1px solid var(--line);
      border-radius: 16px;
      background: var(--card);
      box-shadow: 0 5px 18px #2233220a;
    }
    .art {
      display: grid;
      place-items: center;
      height: 125px;
      color: #fff;
      background: linear-gradient(135deg, var(--accent), #d9b79a);
      font-size: 34px;
    }
    .body { padding: 16px; }
    .body h3 { margin: 0 0 5px; font-size: 17px; }
    .body p { margin: 3px 0; color: var(--muted); font-size: 13px; }
    .row {
      display: flex;
      align-items: center;
      justify-content: space-between;
      gap: 10px;
      margin-top: 14px;
    }
    .price { color: var(--accent); font-weight: 800; }
    .action { border-radius: 9px; padding: 9px 12px; }
    .notice { min-height: 24px; margin: 16px 2px; color: var(--accent); font-weight: 700; }
    footer {
      margin-top: 35px;
      padding: 18px 4px;
      border-top: 1px solid var(--line);
      color: var(--muted);
      font-size: 12px;
    }
    @media (max-width: 600px) {
      header { align-items: flex-start; flex-wrap: wrap; }
      nav { order: 3; width: 100%; gap: 12px; flex-wrap: wrap; }
      main { margin: 22px auto; }
      .hero { padding: 24px; }
    }
  </style>
</head>
<body>
  <header>
    <div class="brand">TECHSAVVY</div>
    <nav><span>Explore</span><span>Services</span><span>About</span></nav>
    <button class="cart" id="cartButton">My selections (0)</button>
  </header>
  <main>
    <section class="hero">
```

```
      <div class="eyebrow">Internet Programming · Experiment 09</div>
      <h1>Online Electronics Shopping</h1>
      <p>Find your next upgrade</p>
    </section>
    <section class="search" aria-label="Search and filters">
      <input id="searchBox" type="search" placeholder="Search by name or detail" aria-label="Search">
      <select id="category">
        <option value="">All options</option>
        <option value="popular">Popular</option>
        <option value="available">Available now</option>
      </select>
      <button class="action" id="searchButton">Search</button>
    </section>
    <div class="sectionhead">
      <h2>Recommended for you</h2>
      <span id="countLabel">3 options</span>
    </div>
    <section class="grid" id="catalog" aria-live="polite"></section>
    <div class="notice" id="notice" role="status">Choose an option to continue.</div>
    <footer>
      Demonstration interface with sample data. Authentication, live availability, payments and
      notifications require a server integration.
    </footer>
  </main>
  <script>
    const items = [
      {
        name: "Nova X Smartphone",
        line: "128 GB · 5G · 6.5 in",
        meta: "4.8 ★ · In stock",
        price: "■28,999"
      },
      {
        name: "AeroBook 14",
        line: "16 GB RAM · 512 GB SSD",
        meta: "4.7 ★ · In stock",
        price: "■64,990"
      },
      {
        name: "Pulse Buds Pro",
        line: "Noise cancellation · 30 h",
        meta: "4.6 ★ · In stock",
        price: "■5,499"
      }
    ];
    let selected = 0;
    const catalog = document.querySelector('#catalog');
    const notice = document.querySelector('#notice');

    function render(list = items) {
      catalog.innerHTML = '';
      document.querySelector('#countLabel').textContent = `${list.length} options`;
      list.forEach((item, index) => {
        const card = document.createElement('article');
        card.className = 'card';
        card.innerHTML = `
          <div class="art" aria-hidden="true">${['❖', '■', '✧'][index % 3]}</div>
          <div class="body">
            <h3>${item.name}</h3>
            <p>${item.line}</p>
            <p>${item.meta}</p>
            <div class="row">
              <span class="price">${item.price}</span>
              <button class="action" data-index="${index}">Select</button>
            </div>
          </div>`;
        catalog.append(card);
      });
    }

    document.querySelector('#searchButton').addEventListener('click', () => {
      const query = document.querySelector('#searchBox').value.trim().toLowerCase();
      const matches = items.filter(item =>
        `${item.name} ${item.line} ${item.meta}`.toLowerCase().includes(query)
      );
      render(matches);
      notice.textContent = query ? 'Search results updated.' : 'Showing all options.';
    });

    catalog.addEventListener('click', event => {
      const button = event.target.closest('button[data-index]');
      if (!button) return;
      const chosen = catalog.querySelectorAll('button[data-index]')[Number(button.dataset.index)];
      selected += 1;
      document.querySelector('#cartButton').textContent = `My selections (${selected})`;
      notice.textContent = `${chosen.closest('.body').querySelector('h3').textContent} added to your
selections.`;
    });

    document.querySelector('#cartButton').addEventListener('click', () => {
      notice.textContent = selected
        ? `You have ${selected} item(s) selected. Review your request.`
        : 'Your selections are empty. Choose an option first.';
    });

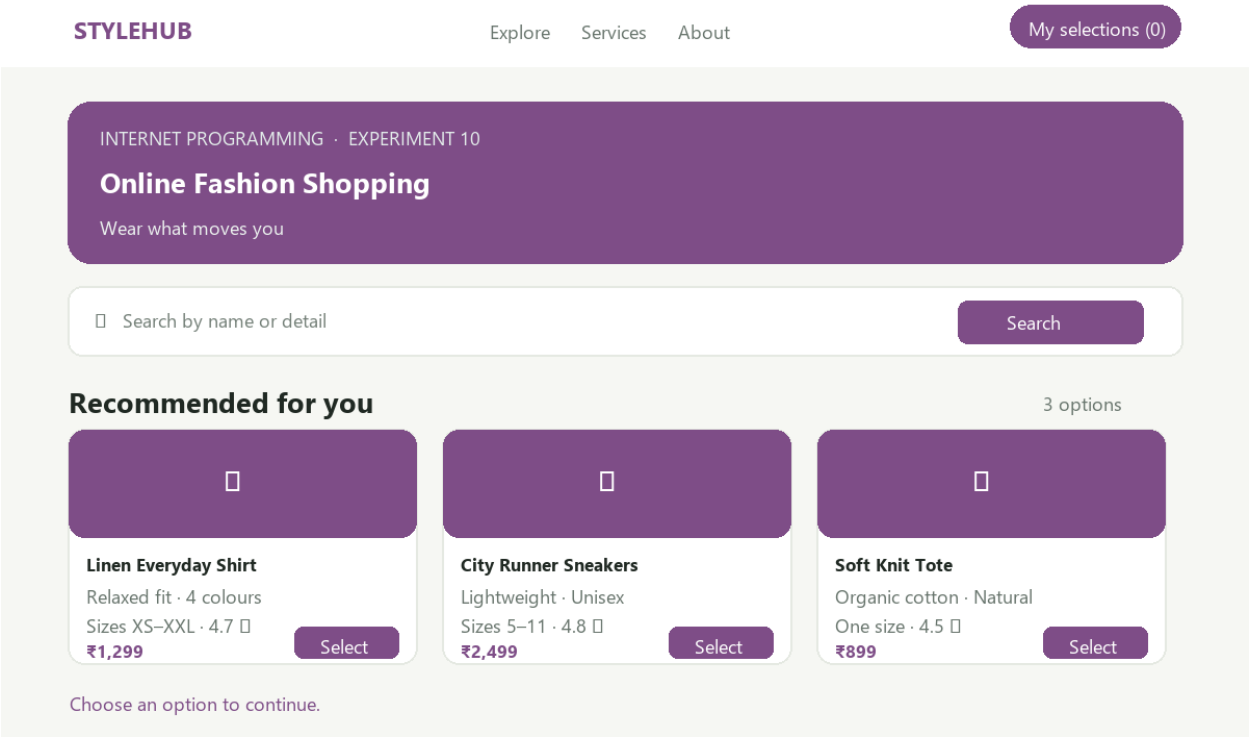
    render();
  </script>
```

```
</body>
</html>
```

Experiment 10: Online Fashion Shopping

Aim: To create a fashion storefront with product search, category filtering, product details and an interactive sample cart, including style and size descriptions.

Output screenshot



Result: The clothing and accessories catalogue was displayed with sample sizes and ratings, and users can search and select an item.

Complete source code (standalone HTML)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Online Fashion Shopping</title>
  <style>
    :root {
      --ink: #202923;
      --muted: #68756d;
      --paper: #f6f7f3;
      --card: #fff;
      --line: #e5e9e2;
      --accent: #7e4d87;
    }
    * { box-sizing: border-box; }
    body {
      margin: 0;
      background: var(--paper);
      color: var(--ink);
      font: 15px/1.5 system-ui, -apple-system, "Segoe UI", sans-serif;
    }
    header {
      display: flex;
      align-items: center;
      justify-content: space-between;
      gap: 20px;
      padding: 16px max(5vw, 20px);
      background: #fff;
      border-bottom: 1px solid var(--line);
    }
    .brand { color: var(--accent); font-size: 18px; font-weight: 800; letter-spacing: .12em; }
    nav { display: flex; gap: 22px; color: var(--muted); font-size: 14px; }
    .cart, .action {
```

```
      border: 0;
      background: var(--accent);
      color: #fff;
      font-weight: 700;
      cursor: pointer;
    }
    .cart { border-radius: 99px; padding: 10px 16px; }
    main { max-width: 1100px; margin: 40px auto; padding: 0 22px; }
    .hero {
      padding: 30px 32px;
      border-radius: 22px;
      color: #fff;
      background: linear-gradient(120deg, var(--accent), #273744);
    }
    .eyebrow { text-transform: uppercase; letter-spacing: .14em; font-size: 11px; opacity: .8; }
    h1 { margin: 7px 0 8px; font-size: clamp(28px, 4vw, 42px); line-height: 1.1; }
    .hero p { margin: 0; opacity: .88; }
    .search {
      display: flex;
      flex-wrap: wrap;
      gap: 10px;
      margin: 20px 0 24px;
      padding: 14px;
      border: 1px solid var(--line);
      border-radius: 16px;
      background: #fff;
    }
    input, select {
      min-width: 150px;
      flex: 1;
      padding: 11px 12px;
      border: 1px solid var(--line);
      border-radius: 10px;
      color: var(--ink);
      background: #fff;
      font: inherit;
    }
    .sectionhead {
      display: flex;
      align-items: end;
      justify-content: space-between;
      margin: 26px 0 14px;
    }
    h2 { margin: 0; font-size: 22px; }
    .sectionhead span { color: var(--muted); font-size: 13px; }
    .grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(245px, 1fr)); gap: 16px; }
    .card {
      overflow: hidden;
      border: 1px solid var(--line);
      border-radius: 16px;
      background: var(--card);
      box-shadow: 0 5px 18px #2233220a;
    }
    .art {
      display: grid;
      place-items: center;
      height: 125px;
      color: #fff;
      background: linear-gradient(135deg, var(--accent), #d9b79a);
      font-size: 34px;
    }
    .body { padding: 16px; }
    .body h3 { margin: 0 0 5px; font-size: 17px; }
    .body p { margin: 3px 0; color: var(--muted); font-size: 13px; }
    .row {
      display: flex;
      align-items: center;
      justify-content: space-between;
      gap: 10px;
      margin-top: 14px;
    }
    .price { color: var(--accent); font-weight: 800; }
    .action { border-radius: 9px; padding: 9px 12px; }
    .notice { min-height: 24px; margin: 16px 2px; color: var(--accent); font-weight: 700; }
    footer {
      margin-top: 35px;
      padding: 18px 4px;
      border-top: 1px solid var(--line);
      color: var(--muted);
      font-size: 12px;
    }
    @media (max-width: 600px) {
      header { align-items: flex-start; flex-wrap: wrap; }
      nav { order: 3; width: 100%; gap: 12px; flex-wrap: wrap; }
      main { margin: 22px auto; }
      .hero { padding: 24px; }
    }
  </style>
</head>
<body>
  <header>
    <div class="brand">STYLEHUB</div>
    <nav><span>Explore</span><span>Services</span><span>About</span></nav>
    <button class="cart" id="cartButton">My selections (0)</button>
  </header>
  <main>
    <section class="hero">
```

```
      <div class="eyebrow">Internet Programming · Experiment 10</div>
      <h1>Online Fashion Shopping</h1>
      <p>Wear what moves you</p>
    </section>
    <section class="search" aria-label="Search and filters">
      <input id="searchBox" type="search" placeholder="Search by name or detail" aria-label="Search">
      <select id="category">
        <option value="">All options</option>
        <option value="popular">Popular</option>
        <option value="available">Available now</option>
      </select>
      <button class="action" id="searchButton">Search</button>
    </section>
    <div class="sectionhead">
      <h2>Recommended for you</h2>
      <span id="countLabel">3 options</span>
    </div>
    <section class="grid" id="catalog" aria-live="polite"></section>
    <div class="notice" id="notice" role="status">Choose an option to continue.</div>
    <footer>
      Demonstration interface with sample data. Authentication, live availability, payments and
      notifications require a server integration.
    </footer>
  </main>
  <script>
    const items = [
      {
        name: "Linen Everyday Shirt",
        line: "Relaxed fit · 4 colours",
        meta: "Sizes XS–XXL · 4.7 ★",
        price: "■1,299"
      },
      {
        name: "City Runner Sneakers",
        line: "Lightweight · Unisex",
        meta: "Sizes 5–11 · 4.8 ★",
        price: "■2,499"
      },
      {
        name: "Soft Knit Tote",
        line: "Organic cotton · Natural",
        meta: "One size · 4.5 ★",
        price: "■899"
      }
    ];
    let selected = 0;
    const catalog = document.querySelector('#catalog');
    const notice = document.querySelector('#notice');

    function render(list = items) {
      catalog.innerHTML = '';
      document.querySelector('#countLabel').textContent = `${list.length} options`;
      list.forEach((item, index) => {
        const card = document.createElement('article');
        card.className = 'card';
        card.innerHTML = `
          <div class="art" aria-hidden="true">${['❖', '■', '✧'][index % 3]}</div>
          <div class="body">
            <h3>${item.name}</h3>
            <p>${item.line}</p>
            <p>${item.meta}</p>
            <div class="row">
              <span class="price">${item.price}</span>
              <button class="action" data-index="${index}">Select</button>
            </div>
          </div>`;
        catalog.append(card);
      });
    }

    document.querySelector('#searchButton').addEventListener('click', () => {
      const query = document.querySelector('#searchBox').value.trim().toLowerCase();
      const matches = items.filter(item =>
        `${item.name} ${item.line} ${item.meta}`.toLowerCase().includes(query)
      );
      render(matches);
      notice.textContent = query ? 'Search results updated.' : 'Showing all options.';
    });

    catalog.addEventListener('click', event => {
      const button = event.target.closest('button[data-index]');
      if (!button) return;
      const chosen = catalog.querySelectorAll('button[data-index]')[Number(button.dataset.index)];
      selected += 1;
      document.querySelector('#cartButton').textContent = `My selections (${selected})`;
      notice.textContent = `${chosen.closest('.body').querySelector('h3').textContent} added to your
selections.`;
    });

    document.querySelector('#cartButton').addEventListener('click', () => {
      notice.textContent = selected
        ? `You have ${selected} item(s) selected. Review your request.`
        : 'Your selections are empty. Choose an option first.';
    });

    render();
  </script>
```

```
</body>
</html>
```